MCGI: Manifold-Consistent Graph Indexing for Billion-Scale Disk-Resident Vector Search

Dongfang Zhao
University of Washington, Tacoma School of Engineering and Technology
dzhao@uw.edu

Abstract

Graph-based Approximate Nearest Neighbor (ANN) search often suffers from performance degradation in high-dimensional spaces due to the Euclidean-Geodesic mismatch, where greedy routing diverges from the underlying data manifold. To address this challenge, this paper presents Manifold-Consistent Graph Indexing (MCGI), a geometry-aware and disk-resident indexing method that leverages Local Intrinsic Dimensionality (LID) to allocate the search effort of a graph index according to the intrinsic geometry of data. Specifically, MCGI estimates the LID around each query from the distances that its own search has already computed and resizes the candidate list of the query once, so that queries in high-LID regions receive longer lists while queries in low-LID regions terminate earlier; the estimate itself costs no additional disk reads. Theoretical analysis shows that the success probability of any fixed search list decays exponentially with the LID of the query. At matched high recall, MCGI raises the query throughput of DiskANN on the same standard Vamana index by up to $1.43\times$ on three million-scale benchmarks and by up to $1.24\times$ on the billion-scale SIFT1B and Deep1B.

1 Introduction

The advent of Large Language Models [5, 37] has fundamentally transformed the landscape of information retrieval and knowledge management. To address the inherent limitations of LLMs, such as hallucinations [20] and knowledge cutoff dates, Retrieval-Augmented Generation (RAG) [27] has emerged as a critical architectural paradigm. RAG relies heavily on the ability to retrieve semantically relevant context from massive corpora in real-time, typically via dense vector representations [25]. This dependency has placed Approximate Nearest Neighbor Search (ANNS) at the core of modern data infrastructure, demanding vector indices that can scale to billion-point datasets [33] while maintaining low latency and high recall under strict production constraints.

For extreme-scale embedding vector applications, many ANNS techniques have noticeably converged on graph-based indices, with DiskANN (Vamana) and its variants [34, 12, 13] being representative solutions for large-scale, SSD-resident workloads. These algorithms typically employ greedy routing on a proximity graph to navigate from an entry point to the query target. While such methods exhibit exceptional performance on popular benchmarks like SIFT1M [22] (128 dimensions), their efficiency degrades significantly in high-dimensional spaces, such as GIST1M (960 dimensions). This degradation is often attributed to the curse of dimensionality [3], where the distance contrast diminishes, and the Euclidean shortest path on the graph diverges from the geodesic path on the underlying data manifold [35, 19]. We refer to this phenomenon as the *Euclidean-Geodesic Mismatch*, or EGM. When the routing algorithm ignores the intrinsic geometry of the data, it performs excessive backtracking and disk I/O, rendering the search inefficient, which is particularly problematic in high-recall scenarios where production systems often expect Recall@10 higher than 95%.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Our observation on this EGM problem is that high-dimensional real-world data is rarely uniformly distributed; Instead, it typically adheres to the Manifold Hypothesis [35, 32], residing on lower-dimensional structures embedded within the ambient space. Consequently, the search difficulty is not uniform across the dataset but is modulated by the Local Intrinsic Dimensionality (LID) [26]. That is, in regions where the data manifold is flat (low LID), greedy routing is effective; however, in regions with high curvature or complex topology (high LID), standard greedy strategies may fail to identify the descent direction. *We thus argue that an optimal indexing strategy must be manifold-aware, dynamically allocating computational resources based on the local geometric complexity.*

To demonstrate the above hypothesis, we design Manifold-Consistent Graph Indexing (MCGI), a geometry-aware disk-based indexing method that allocates the resources of a graph index according to the LID of the data. During search, MCGI estimates the LID around each query from the distances that its search has computed after a few tens of expansions, and it resizes the candidate list of the query once, so that queries in high-LID regions receive longer lists and the other queries stop earlier. This search-time adaptation changes neither the index format nor the I/O path of the search; consequently, it applies to any Vamana index and removes disk reads that a fixed list would spend on easy queries. The same rule can also set the pruning parameter of each node during construction, where a higher LID relaxes the occlusion test of the node.

The remainder of this paper will present the following contributions: (1) We establish a theoretical framework that relates the local intrinsic dimensionality to graph navigability, with formal guarantees on estimation robustness, fixed-beam suboptimality, and construction-time adaptation; (2) We design geometry-aware algorithms that apply one LID-driven allocation rule at both stages of a disk-resident graph index, namely a query-adaptive candidate list that sizes the search of each query by the LID around it and, as an optional construction-time component, a pruning parameter that relaxes the occlusion test of high-LID nodes; and (3) We evaluate MCGI on five benchmarks and show up to $1.33\times$, $1.43\times$, and $1.20\times$ higher throughput than DiskANN at matched high recall on SIFT1M, GloVe-100, and GIST1M, respectively, up to $1.22\times$ and $1.24\times$ on one billion vectors of SIFT1B and Deep1B, respectively, and 13–18% fewer disk reads per query than DiskANN and PipeANN on GIST1M.

2 Related Work

Vector Indexing Paradigms While traditional sparse retrieval methods rely on lexical matching [31], memory-resident graph indices like HNSW [29] establish state-of-the-art dense retrieval but face billion-scale memory bottlenecks. Disk-based approaches, such as DiskANN [34], SPANN [6], PipeANN [12], and OdinANN [13], along with predecessors like NSG [10], adapt topologies or inverted structures for SSDs. Recent advancements such as CSPG [39] further optimize sparse proximity graphs to reduce unpromising explorations, yet these methods predominantly rely on static routing parameters evaluated on moderate dimensionality. In contrast, MCGI dynamically modulates the search with local topology to align graph traversal with the underlying manifold.

High-Dimensional Indexing Early space-partitioning trees like the KD-tree [4] and R-tree [15] severely degrade to linear-scan performance in dimensions higher than 20 due to the curse of dimensionality. Approximate methods address this through sub-linear hashing like LSH [18, 8], which incurs excessive storage overhead, or subspace quantization like PQ [22] and OPQ [11], and randomized structures like RP-Trees [7], which sacrifice accuracy through compression. Furthermore, recent literature extensively studies the theoretical limits of navigable graphs [9] and the efficiency of index construction [41] in strictly high-dimensional scenarios. MCGI keeps the graph traversal of a disk-resident index, which uses compressed codes only to order candidates and full-precision vectors from disk for the final distances, and adapts the length of this traversal to the local topology of each query.

Intrinsic Dimensionality Theoretical metrics such as the doubling dimension [14] and expansion dimension [24] provide asymptotic bounds on search complexity, and recent analyses establish worst-case performance limits for popular implementations like DiskANN and HNSW based on intrinsic dimensionality [19]. Maximum likelihood estimators [26] for Local Intrinsic Dimensionality (LID) enable robust practical estimations. Existing works typically utilize LID passively for query hardness prediction [16, 38] or detecting adversarial examples [28] without altering the underlying

index structure. Diverging from passive descriptive applications, MCGI uses the LID of each node and of each query as a control quantity, which sets the pruning parameter during construction and the candidate-list size during search.

3 Methodology

MCGI allocates the resources of a disk-resident graph index according to the local intrinsic dimensionality (LID) of the data, because greedy routing needs more steps to find a descent direction where the LID around the target is high. The primary application of this rule is the search, where a query with a high LID receives a longer candidate list $L(q)$, which spends disk reads where a fixed list fails and saves them where the search converges early. The same rule optionally applies to construction, where a node with a high LID receives a larger pruning parameter $\alpha(u)$ and therefore a more permissive occlusion test for its candidate edges. Under the degree bound R of the index, this construction-time rule changes which candidate edges a node keeps rather than how many, whereas the search-time rule applies to any Vamana index regardless of how the index was built.

3.1 Mapping Function from LID to Pruning Parameter

We start by briefly reviewing the notions of Local Intrinsic Dimensionality (LID) in the words of analysis. A full treatment can be found, for example, in Houle [17].

Definition 3.1 (Local Intrinsic Dimensionality)**.** Let $\mathcal{X}$ be a domain equipped with a distance measure $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^+$. For a reference point $x \in \mathcal{X}$, let $F_x(r) = \mathbb{P}(d(x, Y) \leq r)$ denote the cumulative distribution function (CDF) of the distance between x and a random variable Y drawn from the underlying data distribution, where $\mathbb{P}$ denotes the probability function and r denotes the radius. The Local Intrinsic Dimensionality (LID) of x, denoted as $\mathrm{LID}(x)$, is defined as the intrinsic growth rate of the probability measure within the neighborhood of x:

$$\mathrm{LID}(x) \triangleq \lim_{r \to 0} \frac{r \cdot F'_x(r)}{F_x(r)} = \lim_{r \to 0} \frac{d \ln F_x(r)}{d \ln r}, \tag{1}$$

provided that the limit exists and $F_x(r)$ is continuously differentiable for $r > 0$.

While Definition 3.1 is elegant, it is often impractical to compute directly. According to [1], LID can be estimated as follows.

Definition 3.2 (LID Maximum Likelihood Estimator)**.** Given a reference point x and its k-nearest neighbors determined by the distance measure d. Let $r_i = d(x, v_i)$ denote the distance to the i-th nearest neighbor, sorted such that $r_1 \leq \cdots \leq r_k$. Following the formulation in [1], which adapts the Hill estimator for intrinsic dimensionality, the LID at x can be estimated as:

$$\widehat{\mathrm{LID}}(x) = -\left(\frac{1}{k} \sum_{i=1}^{k} \ln \frac{r_i}{r_k} \right)^{-1}. \tag{2}$$

The primary hypothesis of the proposed Manifold-Consistent Graph Indexing (MCGI) is that the graph topology should adapt to the local geometric complexity, rather than being a static vertex-edge structure. To preserve topological fidelity in complex regions, the indexing algorithm must adopt a conservative pruning strategy there (larger α in [34]), under which more candidate edges of each node survive the occlusion test before the degree bound R applies. The question then becomes how to quantitatively map the estimated LID to the pruning parameter α that controls the edge selection during graph construction. We refer to such a function as the *mapping function* $\Phi : \mathbb{R}^+ \to [\alpha_{\min}, \alpha_{\max}]$, which translates the geometric complexity of the local neighborhood into a corresponding pruning constraint.

Formally, let $u \in V$ be a node in the graph and $\widehat{\mathrm{LID}}(u)$ be its estimated LID. We define the *pruning parameter* $\alpha(u)$ as:

$$\alpha(u) \triangleq \Phi(\widehat{\mathrm{LID}}(u)), \tag{3}$$

where Φ will be constructed to satisfy the following geometric requirements: in regions with high LID, the graph should prune conservatively (larger α) to retain the navigational edges that greedy

routing needs; conversely, in low-LID regions, where greedy routing finds descent directions easily, the pruning can be aggressive (smaller α) and keep the graph sparse.

To ensure the mapping is robust across datasets with varying complexity scales, we employ Z-score normalization based on the empirical distribution of the LID estimates. We first compute the normalized score $z(u)$:

$$z(u) = \frac{\widehat{\text{LID}}(u) - \mu_{\widehat{\text{LID}}}}{\sigma_{\widehat{\text{LID}}}}, \tag{4}$$

where $\mu_{\widehat{\text{LID}}}$ and $\sigma_{\widehat{\text{LID}}}$ denote the mean and standard deviation of the set of estimated LID values $\{\widehat{\text{LID}}(v) \mid v \in V\}$ computed across the entire graph. We then formulate Φ using a logistic function to smoothly map the Z-score to the operational range $[\alpha_{\min}, \alpha_{\max}]$:

$$\Phi(\widehat{\text{LID}}(u)) = \alpha_{\min} + \frac{\alpha_{\max} - \alpha_{\min}}{1 + \exp(-z(u))}. \tag{5}$$

The mapping function Φ satisfies the following geometric properties that are essential for stable graph construction: Monotonicity and Boundedness. We state these properties in the following as propositions with proof sketches and provide full proofs in the Appendix.

Proposition 3.3 (Monotonicity)**.** *The mapping function Φ is strictly increasing with respect to the estimated local intrinsic dimensionality. Formally, given that the standard deviation of the LID estimates $\sigma_{\widehat{LID}} > 0$ and the pruning range $\alpha_{\max} > \alpha_{\min}$, the derivative satisfies:*

$$\frac{d\Phi}{d\widehat{LID}(u)} > 0. \tag{6}$$

Proof Sketch. By the chain rule, $\frac{d\Phi}{d\widehat{\text{LID}}(u)} = \frac{d\Phi}{dz} \cdot \frac{dz}{d\widehat{\text{LID}}(u)}$, where the Z-score satisfies $\frac{dz}{d\widehat{\text{LID}}(u)} = \frac{1}{\sigma_{\widehat{\text{LID}}}} > 0$. The derivative of the logistic component evaluates to $\frac{d\Phi}{dz} = (\alpha_{\max} - \alpha_{\min}) \cdot \frac{e^{-z}}{(1+e^{-z})^2}$, which is strictly positive since $\alpha_{\max} > \alpha_{\min}$ and $e^{-z} > 0$ for all $z \in \mathbb{R}$. Combining these two terms yields a strictly positive derivative: $\alpha(u)$ increases monotonically with the geometric complexity. □

Proposition 3.4 (Boundedness)**.** *The pruning parameter $\alpha(u)$ derived from the mapping function is strictly bounded within the prescribed operational interval. That is, for any node u with a finite LID estimate, the following holds:*

$$\alpha_{\min} < \alpha(u) < \alpha_{\max}. \tag{7}$$

Proof Sketch. For any finite $\widehat{\text{LID}}(u)$, the Z-score $z(u)$ is finite, and the logistic function satisfies $S(u) = \frac{1}{1+e^{-z(u)}} \in (0, 1)$ strictly, since $e^{-z(u)} \in (0, \infty)$. Substituting into $\alpha(u) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot S(u)$ and applying the strict bounds on $S(u)$ directly yields $\alpha_{\min} < \alpha(u) < \alpha_{\max}$. □

3.2 Indexing Algorithms

The Manifold-Consistent Graph Indexing (MCGI) algorithm introduces a geometric calibration phase to a graph indexing procedure. Unlike static methods that apply a uniform connectivity rule, MCGI executes in two distinct stages to ensure that the topology respects the manifold structure, as follows. The full pseudo-code of the algorithm is in Appendix A.4, Algorithm 1.

Phase 1: Geometric Calibration. Before modifying the graph topology, the system first performs a global analysis of the dataset geometry. We estimate the LID for every point and aggregate the population statistics (μ, σ) defined in Section 3.1. This phase freezes the geometric profile of the dataset. By pre-computing these statistics, we decouple the complexity estimation from the graph update loop, ensuring that the mapping function Φ remains stable and computationally efficient during the intensive edge-selection process.

Phase 2: Manifold-Consistent Refinement. The index construction follows an iterative refinement strategy. Let $\mathcal{N}(u)$ denote the set of neighbors for node u in the graph G. In each iteration, the algorithm dynamically updates $\mathcal{N}(u)$ by considering a candidate pool $\mathcal{C}$ obtained from a greedy search on the current graph structure:

1. Queries the pre-computed geometric profile to determine the node-specific constraint $\alpha(u)$.
2. Explores the graph to identify a candidate pool $\mathcal{C}$.
3. Filters connections using the dynamic occlusion criterion.

Remark The calibration phase estimates the LID of each point from its approximate k nearest neighbors in $O(N \log N)$ time, which the construction phase dominates, on par with the complexity of DiskANN [34]. In practice, calculating the exact LID for the entire dataset may cause a computational bottleneck on billion-scale data. To address this, we also design an online algorithm called *Online-MCGI*, whose pseudo-code can be found in Appendix A.4, Algorithm 2.

3.3 Theoretical Analysis

3.3.1 Geometric Complexity and Adaptive Routing

The efficiency of greedy routing on a proximity graph is intrinsically tied to the local topological properties of the underlying manifold $\mathcal{M}$. Under the Manifold Hypothesis, the search space is locally characterized by the Local Intrinsic Dimensionality (LID) at query q. We formalize the relationship between LID and the computational cost required to identify the nearest neighbor, as follows.

Lemma 3.5 (Local Complexity Lower Bound)**.** *Consistent with the complexity bounds established for growth-restricted metrics [24], for a query q on a manifold $\mathcal{M}$, the expected number of distance evaluations N_{dist} required for successful greedy routing scales exponentially with the local intrinsic dimensionality $d = \textit{LID}(q)$:*

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\sqrt{d} \cdot \exp(\lambda \cdot d)\right), \tag{8}$$

where $\lambda > 0$ is a geometric constant derived from the required convergence rate.

Proof Sketch. At a node u with distance r to query q, a greedy step succeeds only if a neighbor v falls within a cone of half-angle $\theta_{\max}$ around the direction of q; the probability of this event corresponds to the ratio of a spherical cap to the full hypersphere $\mathcal{S}^{d-1}$. Asymptotic analysis of this ratio via the regularized incomplete beta function yields $P_{\text{success}} \propto d^{-1/2} \exp(-\lambda \cdot d)$, where the exponential decay arises from the concentration of measure on high-dimensional spheres and $\lambda = -\ln(\sin \theta_{\max}) > 0$. Since $\mathbb{E}[N_{dist}] = 1/P_{\text{success}}$, the lower bound follows directly. □

Lemma 3.5 reveals a fundamental geometric barrier: for a fixed budget L, the probability of routing failure grows exponentially in high-LID regions. To address this, MCGI adopts an *Iso-Recall* strategy, aiming to homogenize the search reliability across the heterogeneous manifold.

Proposition 3.6 (Optimal Budget Allocation)**.** *To maintain a uniform bound on the routing failure probability δ across the manifold (i.e., $\mathbb{P}(\textit{fail} \mid q) \leq \delta$ for all $q \in \mathcal{M}$), the search beam width $L(q)$ must scale exponentially with the local intrinsic dimensionality:*

$$L(q) \propto \exp\left(\lambda \cdot \textit{LID}(q)\right). \tag{9}$$

Proof Sketch. For a beam of width L, the failure probability is $(1 - P_{\text{success}})^L \leq \delta$, which via the approximation $\ln(1 - x) \approx -x$ requires $L \geq -\ln \delta / P_{\text{success}}$. Substituting the bound $P_{\text{success}} \propto d^{-1/2} \exp(-\lambda \cdot d)$ from Lemma 3.5 and absorbing the polynomial and confidence factors into a constant C_δ yields $L(q) \geq C_\delta \cdot \exp(\lambda \cdot \text{LID}(q))$. □

Proposition 3.6 implies that the search budget of each query should grow with the LID of its neighborhood. A change of L at every expansion step would disrupt memory alignment and the vectorized distance computation of the search loop; in contrast, a single resize of the candidate list per query leaves both intact. MCGI therefore realizes the budget of Proposition 3.6 at query time through the query-adaptive search list of Section 3.4, and it can complement this budget at construction time through the pruning parameter $\alpha(u)$.

3.3.2 Topological Fidelity and Connectivity

A primary theoretical concern with aggressive edge pruning is the potential fracture of the connectivity backbone [40]. We show that, when the occlusion test runs over all points and no degree bound applies, the pruning of MCGI preserves every edge of the Euclidean minimum spanning tree and therefore global reachability.

Formally, the edge selection in MCGI is governed by the adaptive pruning parameter $\alpha(u)$. Specifically, an edge (u, v) is pruned if there exists a witness node n such that $\alpha(u) \cdot d(n, v) \leq d(u, v)$. This condition defines an *exclusion region* around the midpoint of u and v. As $\alpha(u)$ increases, this region shrinks, making pruning more conservative. We formulate this in the following proposition.

Proposition 3.7 (Connectivity Preservation)**.** *Let G_{EMST} and G_{RNG} denote the Euclidean Minimum Spanning Tree (EMST) and the Relative Neighborhood Graph (RNG), respectively. For any configuration of points in general position, provided that $\alpha(u) \geq 1.0$ for all $u \in V$, that the occlusion test of each node runs over all other points, and that no degree bound truncates the neighbor lists, the graph G_{MCGI} satisfies the inclusion hierarchy:*

$$E_{EMST} \subseteq E_{RNG} \subseteq E_{MCGI}. \tag{10}$$

Consequently, G_{MCGI} is connected.

Proof Sketch. The first inclusion $E_{EMST} \subseteq E_{RNG}$ follows from the classical result of Toussaint [36], which holds for any finite point set in a metric space. For the second inclusion $E_{RNG} \subseteq E_{MCGI}$, note that MCGI prunes an edge (u, v) only if a witness node falls within a exclusion region of radius $\frac{1}{\alpha(u)} d(u, v)$ around v; since $\alpha(u) \geq 1.0$, this exclusion region is a strict geometric subset of the RNG exclusion region, so any edge surviving the RNG pruning condition is guaranteed to survive MCGI pruning. Connectivity of G_{MCGI} follows immediately from the inclusion hierarchy, since G_{EMST} spans all vertices by definition. □

Proposition 3.7 ensures that there are no structural dead ends. While Lemma 3.5 dictates the *cost* of routing, Proposition 3.7 ensures the *viability* of routing. Even in the worst-case scenario where heuristics fail, a path exists from the entry point to any target via the edges of the underlying EMST, provided the graph traversal algorithm has sufficient width to discover these backbone links.

The adaptive pruning also tolerates the estimation noise of the finite-sample estimator in Definition 3.2. Proposition A.6 in Appendix A.1 bounds the induced perturbation of the pruning parameter by $(\alpha_{\max} - \alpha_{\min})\,\mathrm{LID}(u)/(4\,\sigma_{\widehat{\mathrm{LID}}}\sqrt{k})$, because the logistic derivative is at most $1/4$ and the estimation error is of order $\mathrm{LID}(u)/\sqrt{k}$ [26]. Consequently, a modest neighborhood size such as $k = 20$ already yields stable pruning parameters.

3.3.3 Fixed-Beam Suboptimality and Adaptation

Proposition 3.6 establishes that an *ideal* search system allocates the beam width $L^*(q) \propto \exp(\lambda \cdot \mathrm{LID}(q))$ at query time. We now formalize the cost of a fixed beam width, which motivates both the query-adaptive search list of MCGI and its construction-time adaptation.

Definition 3.8 (Iso-Recall Beam Schedule)**.** For a target failure probability δ, the iso-recall beam schedule assigns each query the list size

$$L^*(q) = \frac{\ln(1/\delta)}{P_{\mathrm{success}}(q)} = C_\delta \cdot \exp(\lambda \cdot \mathrm{LID}(q))\,, \tag{11}$$

where the second form follows from Lemma 3.5, and the constant C_δ collects $\ln(1/\delta)$ and the polynomial factor of the lemma, which varies slowly compared with the exponential term. Since $(1 - P_{\mathrm{success}})^L \leq e^{-LP_{\mathrm{success}}}$, the list size $L^*(q)$ keeps the failure probability of q at most δ. A search algorithm is said to be *recall-optimal* if it implements $L^*(q)$ for every query q.

Proposition 3.9 (Suboptimality of Fixed Beam)**.** *For any fixed beam width $\bar{L}$, the success probability of the search at a query q satisfies:*

$$1 - P(\mathrm{fail} \mid q) \;\leq\; \ln(1/\delta)\,\frac{\bar{L}}{L^*(q)} \;=\; \frac{\ln(1/\delta)\,\bar{L}}{C_\delta}\,\exp(-\lambda \cdot \mathrm{LID}(q))\,, \tag{12}$$

which decays exponentially with $\mathrm{LID}(q)$. Consequently, every query with $L^*(q) > \bar{L}\ln(1/\delta)/(1-\delta)$ misses the target success probability $1 - \delta$.

Proof Sketch. By Bernoulli's inequality, the success probability of a beam of width $\bar{L}$ satisfies $1 - (1 - P_{\text{success}}(q))^{\bar{L}} \leq \bar{L}\, P_{\text{success}}(q)$. Substituting $P_{\text{success}}(q) = \ln(1/\delta)/L^*(q)$ from Definition 3.8 yields the first form of Eq. (12), and the exponential form of $L^*(q)$ yields the second. The bound falls below $1 - \delta$ exactly when $L^*(q) > \bar{L} \ln(1/\delta)/(1 - \delta)$. □

Proposition 3.9 explains why static-parameter methods degrade on high-dimensional data: the success probability of a fixed beam decays exponentially with the LID of the query, so that a list long enough for a typical query fails on the queries in high-LID regions. The query-adaptive search list of Section 3.4 reduces this gap directly by approximating $L^*(q)$ for each query. For a search that keeps a fixed beam width, the following corollary establishes that the construction-time adaptation of MCGI applies the same allocation to the edges of each node and monotonically reduces this gap compared with a static pruning parameter, structurally compensating for the shortfall of a constant beam width.

Corollary 3.10 (Construction-time Structural Compensation for Fixed Beam Width)**.** *Under any fixed beam width $\bar{L}$ and without the degree bound R, modulating $\alpha(u)$ during index construction according to $\Phi(\widehat{\text{LID}}(u))$ monotonically reduces $P(\text{fail} \mid q)$ for every query whose beam search visits only high-LID nodes, compared to any static baseline $\bar{\alpha} \leq (\alpha_{\min} + \alpha_{\max})/2$ operating under the same $\bar{L}$. Specifically, nodes in high-LID regions receive conservative pruning (large α), preserving a larger navigational edge set that structurally compensates for the shortfall of $\bar{L}$ relative to the oracle schedule $L^*(q)$. This guarantee is monotone but not tight: it establishes that MCGI dominates fixed-α baselines, rather than quantifying the residual gap to the Iso-Recall Schedule $L^*(q)$ (Definition 3.8) directly.*

Proof Sketch. For nodes where $\text{LID}(u) > \text{LID}_{\text{avg}}$, Proposition 3.3 guarantees $\alpha(u) > \Phi(\mu_{\widehat{\text{LID}}}) = (\alpha_{\min} + \alpha_{\max})/2 \geq \bar{\alpha}$, which shrinks the pruning exclusion region and thus preserves, for the same candidate pool, a superset of edges relative to the static-$\bar{\alpha}$ baseline, i.e., $E_{\text{MCGI}} \supseteq E_{\bar{\alpha}}$ in high-LID regions. A strictly larger outgoing edge set monotonically increases $P_{\text{success}}(q)$ at each beam step by the same geometric argument as Proposition 3.6, yielding $P(\text{fail} \mid q, \text{MCGI}) \leq P(\text{fail} \mid q, \bar{\alpha})$ for every such query q. Note that this guarantee is monotone but not tight: quantifying the residual gap to the iso-recall schedule $L^*(q)$ (Definition 3.8) is left for future work. □

3.4 Query-Adaptive Search List

MCGI applies the iso-recall schedule of Definition 3.8 at query time, where it resizes the candidate list of each query according to the geometry that the search observes around the query. The schedule requires $\text{LID}(q)$ before the search, although the neighborhood of q is unknown at that point. The beam search of a disk-resident Vamana index, however, maintains two structures that expose this neighborhood as the search proceeds. Specifically, the candidate list holds up to $\bar{L}$ nodes ordered by their compressed distances to q, and the expanded set records the exact distance of every node whose vector the search has read from disk. MCGI therefore inspects these two structures at a checkpoint after the search expands n_0 nodes, where n_0 is a small constant of a few tens, and resizes the candidate list once at this checkpoint.

At the checkpoint, MCGI forms a feature vector $\mathbf{f}(q)$ of five quantities that describe the local geometry around q. The first feature is $\ln \widehat{\text{LID}}(q)$, which Definition 3.2 computes from the exact distances $r_1 \leq \cdots \leq r_k$ of the k nearest expanded nodes, where k equals the result size, typically 10. Because k distances yield a noisy estimate, the second feature applies the same estimator to the compressed distances of the m best candidates in the list, where m is a few tens. The next two features, $\ln r_1$ and $\ln r_k$, record the scale of the neighborhood, which the LID estimator removes by construction. The last feature is the logarithm of one plus the number of candidates whose squared distance lies within $(1 + \epsilon)\, r_k^2$ for a small constant ϵ such as 0.2. A large count indicates that many points compete for the last result positions, which is the concentration of distances that a high LID causes (Lemma 3.5).

MCGI maps the features to a list size through a log-linear score that mirrors the exponential form of Eq. (11):

$$L(q) = \min\Big\{ c\bar{L},\ \max\big\{ L_{\min},\ \bar{L} \exp\big(\gamma\, (\mathbf{w}^\top \mathbf{f}(q) + b - \bar{s})\big) \big\} \Big\}, \tag{13}$$

where $\bar{L}$ is the list size that the application requests, $\gamma > 0$ sets the strength of the adaptation, and c is a small cap such as 8. The weights $\mathbf{w}$ and the offset b result from a least-squares fit with a small ridge penalty on a set of calibration queries that is disjoint from the evaluation queries; for each calibration

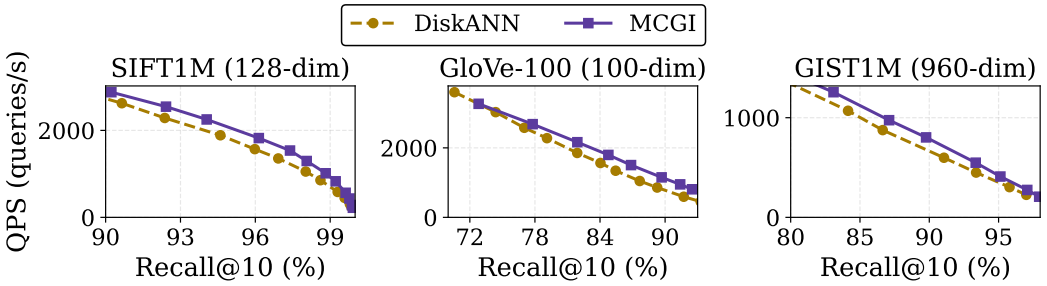


Figure 1: Recall-QPS trade-off of MCGI and DiskANN on the million-scale datasets.

query, the regression target is the logarithm of the smallest list size at which the query reaches its best recall. The constant $\bar{s}$ is the mean score of the calibration queries, so that a query of average geometry keeps the requested list size $\bar{L}$. A query whose features indicate a high LID consequently receives a longer list, whereas a query in a low-LID region terminates earlier and saves the disk reads that a fixed list would spend on it.

The adaptation adds no disk reads, because every feature reuses distances that the search has already computed. Computing $\mathbf{f}(q)$ takes $O(n_0 + \bar{L})$ time once per query, which is small compared with the $O(\bar{L}R)$ compressed-distance evaluations of the search itself. Resizing the list either truncates its tail or raises its capacity, after which the beam search continues from its current state without a restart. The vectorized distance computation therefore remains unchanged, since the adaptation only moves the termination point of each query. Because the index format is also unchanged, the query-adaptive search list applies to any Vamana index, including an index built with the construction-time adaptation of Section 3.2.

4 Evaluation

All experiments run on a cloud virtual machine with 192 virtual cores of Intel Xeon Platinum 8468 and 960 GiB of RAM, where the disk-resident indexes reside on a 17.6 TB virtual block device (virtio). Each search reads graph nodes from this device with direct I/O, and at 32 search threads the measured I/O time is about 0.34 ms per read; consequently, the throughput of DiskANN and MCGI, which share one I/O engine, follows their number of disk reads per query. We evaluate MCGI on three million-scale benchmarks with dimensionality from 100 to 960, namely SIFT1M [22], GloVe-100 [30], and GIST1M [22], and on the billion-scale SIFT1B [23] and Deep1B [2]. The baseline is DiskANN (Vamana) [34], which searches the same index with a fixed candidate list. More setup details are in Appendix A.2.

4.1 Million-scale Effectiveness

We compare MCGI with DiskANN on a single index per dataset, so that the two methods differ only in the candidate list of each query. Each index is built with the standard Vamana pruning rule, and both methods search it with the same compressed vectors and the same requested list sizes $\bar{L}$ from 10 to 400. The weights of Eq. (13) come from 10,000 calibration queries per dataset, which are disjoint from the official evaluation queries. Each method runs once per list size on an otherwise idle machine, and the page cache is dropped before each run.

Figure 1 plots the query throughput against Recall@10 for the two methods. MCGI raises the throughput of DiskANN on all three million-scale datasets, and the improvement grows with the target recall. On SIFT1M, the gain rises from $1.15\times$ at 95% recall to $1.33\times$ at 99%. GloVe-100 and GIST1M follow the same trend, with gains from $1.12\times$ at 80% to $1.43\times$ at 90% and from $1.12\times$ at 85% to $1.20\times$ at 95%, respectively. The growth with recall agrees with Proposition 3.9, because a high target recall forces a fixed search to use a long list for every query, including the many queries in low-LID regions that converge early. MCGI shortens the lists of these queries and lengthens the lists of queries in high-LID regions, which removes disk reads that do not improve the result.

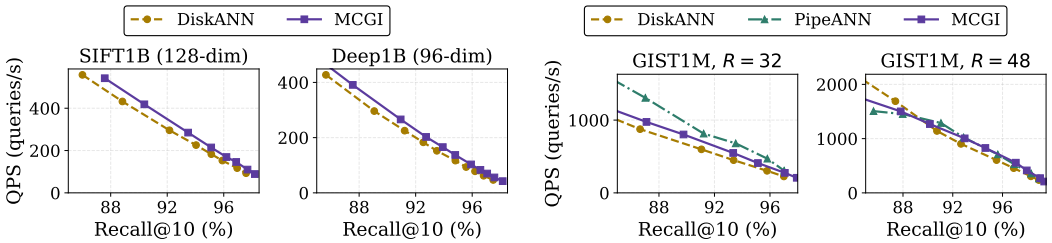


Figure 2: Recall-QPS at one billion vectors (left) and of disk-resident indexes on GIST1M (right).

4.2 Billion-scale Scalability

We build one DiskANN index for each of SIFT1B and Deep1B with the standard sharded construction under a memory budget of 300 GiB. The construction splits the 8-bit vectors of SIFT1B into three overlapping shards and completes in 4.8 hours on the CPU cores of the platform, while the 32-bit vectors of Deep1B need seven shards and 5.4 hours. Both indexes use $R = 32$, $L_{build} = 50$, and 31-byte PQ codes, and both methods again search the same index. The calibration queries of each dataset are the even-indexed half of its 10,000 official queries, and the evaluation uses the odd-indexed half. The requested list sizes extend to $\bar{L} = 1{,}000$ on SIFT1B and to $\bar{L} = 2{,}000$ on Deep1B, since DiskANN needs a list longer than 400 on SIFT1B and longer than 800 on Deep1B to reach 95% recall.

The left two panels of Figure 2 show that the gain of MCGI persists at one billion vectors on both datasets and again grows with the target recall. On SIFT1B, MCGI raises the throughput of DiskANN from $1.07\times$ at 85% recall to $1.17\times$ at 95% and to $1.22\times$ at 97%, where the throughput rises from 115 to 141 queries per second. Deep1B follows the same trend with gains from $1.08\times$ at 85% to $1.18\times$ at 95% and to $1.24\times$ at 97%, where the throughput rises from 58 to 72 queries per second. At the longest list that we evaluate, DiskANN stays below 98% recall on both datasets, whereas MCGI reaches 98.2% on each of them, at 89 queries per second on SIFT1B and 42 on Deep1B.

4.3 Comparison with Pipelined and In-memory Search

We compare MCGI with PipeANN [12, 13], which overlaps the disk reads of each query in a pipelined beam search, on GIST1M with $R = 32$ and $R = 48$. PipeANN searches the same DiskANN index files as DiskANN and MCGI, with 32 threads, its in-memory entry-point index built from a 1% sample, and the better of the I/O widths 4 and 32. HNSW [29], IVF-Flat [21], and CSPG [39] serve as in-memory references, since they keep every full-precision vector in RAM.

Table 1: GIST1M at 95% Recall@10. †In memory.

	$R = 32$		$R = 48$	
Method	QPS	Reads	QPS	Reads
DiskANN	351	272.7	659	146.7
PipeANN	545	268.3	780	143.8
MCGI	420	**223.7**	**785**	**125.5**
CSPG†	5,742	–	9,107	–
HNSW†	6,678	–	–	–
IVF-Flat†	1,292 (no degree bound)			

The right two panels of Figure 2 and Table 1 show that MCGI issues the fewest disk reads of the three disk-resident methods. At 95% recall, MCGI reads 224 nodes per query with $R = 32$ and 126 with $R = 48$, which is 13–18% fewer than DiskANN and PipeANN, whose reads differ from each other by at most 2%. PipeANN turns the same reads into higher throughput by overlapping them, and it leads at $R = 32$ with 545 queries per second against 420 for MCGI. With $R = 48$, MCGI matches PipeANN at 95% recall with 785 against 780 queries per second and surpasses it at 97% recall with 564 against 521. The two techniques act on different costs: PipeANN shortens the wait for each read, while MCGI removes reads, so the query-adaptive list can also size the candidate list of a pipelined search. The in-memory indexes reach 1,292 to 9,107 queries per second at 95% recall, at the price of holding the 3.84 GB of full-precision vectors in RAM, whereas the disk-resident methods hold mainly the 322 MB of PQ codes.

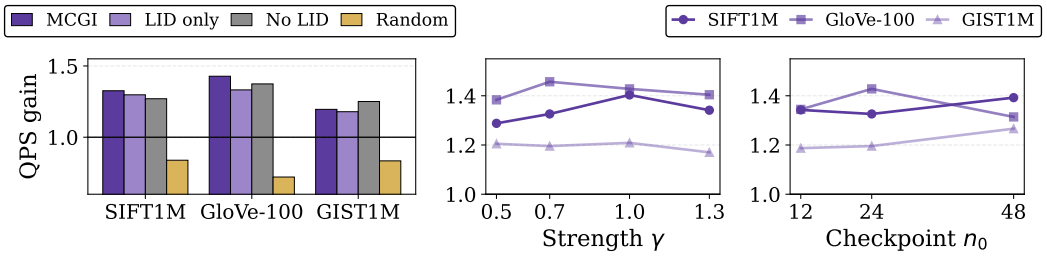


Figure 3: QPS gain of MCGI variants over DiskANN at the highest target recall of each dataset.

4.4 Ablation and Sensitivity

Figure 3 attributes the gain of MCGI at the highest target recall of each million-scale dataset, namely 99% on SIFT1M, 90% on GloVe-100, and 95% on GIST1M. Each variant is fitted on the same calibration queries and searched once on the same index as DiskANN. The random variant keeps the fitted list sizes of MCGI in distribution but assigns them to queries at random, which lowers the throughput to $0.72\times$–$0.84\times$ that of DiskANN. The gain therefore comes from matching each list to the geometry of its query, not from varying the list size. The two LID features alone retain gains of $1.18\times$–$1.33\times$, and the distance features alone, namely the first and tenth expanded distances and the candidate count, retain $1.25\times$–$1.37\times$, since the LID estimate is a function of the same distances.

The last two panels of Figure 3 vary the strength γ of Eq. (13) from 0.5 to 1.3 and the checkpoint n_0 from 12 to 48 expansions around the calibrated setting. Every setting keeps a gain of at least $1.17\times$, and the calibrated setting stays within $0.08\times$ of the best setting on each dataset. The calibration therefore needs no careful tuning of either parameter, and the default $n_0 = 24$ serves all datasets.

5 Conclusion

This paper presents MCGI, a geometry-aware disk-resident indexing method for high-dimensional data of varying intrinsic dimensionality. MCGI addresses the Euclidean-Geodesic mismatch by adapting the candidate list of each query, and optionally the pruning parameter of each node, to local intrinsic dimensionality, where the list size follows from distances that the search has already computed. Our theoretical analysis shows that the success probability of any fixed beam width decays exponentially with the LID of the query, and the query-adaptive list approximates the iso-recall schedule from distances that the search has already computed. At matched high recall, MCGI raises the throughput of DiskANN by up to $1.43\times$ on benchmarks of 96 to 960 dimensions and by up to $1.24\times$ at one billion vectors.

Acknowledgments and Disclosure of Funding

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation. The author appreciates the feedback and praise from early users around the world (e.g., South Korea, New Zealand) of MCGI and AdaDisk (the parent project of MCGI), without which the author would not have been able to continue improving the system and pushing the performance boundary. Copilot and Claude assisted with the system implementation, the experiments, and the writing of this paper; the author verified all results and takes full responsibility for the content.

The source code of MCGI is available at `https://github.com/hpdic/mcgi`.

References

[1] Laurent Amsaleg, Oussama Chelly, Teddy Furon, Stéphane Girard, Michael E. Houle, Ken-ichi Kawarabayashi, and Michael Nett. Estimating local intrinsic dimensionality. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '15, page 29–38, New York, NY, USA, 2015. Association for Computing Machinery.

[2] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2055–2063, 2016.

[3] RICHARD BELLMAN and Stuart Dreyfus. *Dynamic Programming*, volume 33. Princeton University Press, 2010.

[4] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Commun. ACM*, 18(9):509–517, September 1975.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[6] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. Spann: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, 2021. Curran Associates Inc.

[7] Sanjoy Dasgupta and Yoav Freund. Random projection trees and low dimensional manifolds. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, STOC '08, page 537–546, New York, NY, USA, 2008. Association for Computing Machinery.

[8] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, SCG '04, page 253–262, New York, NY, USA, 2004. Association for Computing Machinery.

[9] Haya Diwan, Jinrui Gou, Cameron Musco, Christopher Musco, and Torsten Suel. Navigable graphs for high-dimensional nearest neighbor search: Constructions and limits. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 59513–59531. Curran Associates, Inc., 2024.

[10] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.*, 12(5):461–474, January 2019.

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization. volume 36, page 744–755, USA, April 2014. IEEE Computer Society.

[12] Hao Guo and Youyou Lu. Achieving low-latency graph-based vector search via aligning best-first search algorithm with ssd. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 171–186, Boston, MA, 2025. USENIX Association.

[13] Hao Guo and Youyou Lu. Odinann: Direct insert for consistently stable performance in billion-scale graph-based vector search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, Santa Clara, CA, 2026. USENIX Association.

[14] A. Gupta, R. Krauthgamer, and J.R. Lee. Bounded geometries, fractals, and low-distortion embeddings. In *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings.*, pages 534–543, 2003.

[15] Antonin Guttman. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, SIGMOD '84, page 47–57, New York, NY, USA, 1984. Association for Computing Machinery.

[16] Junfeng He, Sanjiv Kumar, and Shih-Fu Chang. On the difficulty of nearest neighbor search. In *Proceedings of the 29th International Coference on International Conference on Machine Learning*, ICML'12, page 41–48, Madison, WI, USA, 2012. Omnipress.

[17] Michael E. Houle. Local intrinsic dimensionality I: an extreme-value-theoretic foundation for similarity applications. In Christian Beecks, Felix Borutta, Peer Kröger, and Thomas Seidl, editors, *Similarity Search and Applications - 10th International Conference, SISAP 2017, Munich, Germany, October 4-6, 2017, Proceedings*, volume 10609 of *Lecture Notes in Computer Science*, pages 64–79. Springer, 2017.

[18] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, STOC '98, page 604–613, New York, NY, USA, 1998. Association for Computing Machinery.

[19] Piotr Indyk and Haike Xu. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 66239–66256. Curran Associates, Inc., 2023.

[20] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023.

[21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[22] Herve Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[23] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. Searching in one billion vectors: Re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 861–864, 2011.

[24] David R. Karger and Matthias Ruhl. Finding nearest neighbors in growth-restricted metrics. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, STOC '02, page 741–750, New York, NY, USA, 2002. Association for Computing Machinery.

[25] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics.

[26] Elizaveta Levina and Peter Bickel. Maximum likelihood estimation of intrinsic dimension. In L. Saul, Y. Weiss, and L. Bottou, editors, *Advances in Neural Information Processing Systems*, volume 17. MIT Press, 2004.

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[28] Xingjun Ma, Bo Li, Yisen Wang, Sarah M. Erfani, Sudanthi N. R. Wijewickrema, Michael E. Houle, Grant Schoenebeck, Dawn Song, and James Bailey. Characterizing adversarial subspaces using local intrinsic dimensionality. In *International Conference on Learning Representations*, 2018.

[29] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. volume 42, page 824–836, USA, April 2020. IEEE Computer Society.

[30] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, pages 1532–1543. ACL, 2014.

[31] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. In *Foundations and Trends in Information Retrieval*, volume 3, pages 333–389. Now Publishers, Inc., 2009.

[32] Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

[33] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*, pages 177–189. PMLR, 2022.

[34] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. Diskann: fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 2019.

[35] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[36] Godfried T. Toussaint. The relative neighbourhood graph of a finite planar set. *Pattern Recognition*, 12(4):261–268, 1980.

[37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. 2023.

[38] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. Steiner-hardness: A query hardness measure for graph-based ann indexes. *Proc. VLDB Endow.*, 17(13):4668–4682, September 2024.

[39] Ming Yang, Yuzheng Cai, and Weiguo Zheng. Cspg: Crossing sparse proximity graphs for approximate nearest neighbor search. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 103076–103100. Curran Associates, Inc., 2024.

[40] Dongfang Zhao and Li Yang. Incremental isometric embedding of high-dimensional data using connected neighborhood graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(1):86–98, 2009.

[41] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces. *Proc. VLDB Endow.*, 16(8):1979–1991, April 2023.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The paper presents both theoretical and experimental evidence to support its claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Appendix A.3 discusses the scope of the evaluated mechanism, the effect of the degree bound on the construction-time rule, the tail latency of the adaptive list, the calibration requirements, and the modeling assumptions of the theory.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Every proposition states its assumptions, including the absence of a degree bound where it matters, and Appendix A.1 gives the full proofs; Appendix A.3 lists the modeling assumptions of the analysis.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The code, the experiment scripts, and the data behind every figure are available in the public repository `https://github.com/hpdic/mcgi`, and Section 4 and Appendix A.2 describe the setup.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The public repository `https://github.com/hpdic/mcgi` contains the code, the experiment scripts, and the exported results; all datasets are public benchmarks.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 and Appendix A.2 give the datasets, the index and search parameters, the calibration protocol, and the baseline configurations.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Each configuration runs once on an otherwise idle machine with the page cache dropped, and both methods search the same index with the same queries; the paper does not report error bars.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 4 and Appendix A.2 describe the platform (192 virtual cores, 960 GiB of RAM, and a virtual block device), and Table 4 reports the build and calibration times.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We believe we have followed the NeurIPS Code of Ethics in all aspects of our research.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The paper studies the efficiency of nearest-neighbor search over public benchmarks and has no direct societal impact beyond faster retrieval.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no data or model with a risk of misuse.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper cites the creators of every dataset (SIFT1M, GIST1M, SIFT1B, GloVe-100, and Deep1B) and every baseline. DiskANN and Faiss are released under the MIT license, PipeANN and hnswlib under the Apache License 2.0, and CSPG is used from the public repository of its authors for research comparison; the datasets are public benchmarks used under the terms of their original releases.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The code of MCGI is released at `https://github.com/hpdic/mcgi` under the MIT license of its DiskANN code base, with a README that maps every figure and table to its script.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper involves no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs assisted with the implementation, the experiments, and the writing, as stated in the acknowledgments, but they are not a component of the method.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Technical appendices and supplementary material

A.1 Complete Proofs

Proposition A.1 (Monotonicity)**.** *The mapping function Φ is strictly increasing with respect to the estimated local intrinsic dimensionality. Formally, given that the standard deviation of the LID estimates $\sigma_{\widehat{LID}} > 0$ and the pruning range $\alpha_{\max} > \alpha_{\min}$, the derivative satisfies:*

$$\frac{d\Phi}{d\widehat{LID}(u)} > 0. \tag{14}$$

Proof. Let $L = \widehat{\text{LID}}(u)$ be the independent variable. We define the normalized Z-score z as a function of L:

$$z(L) = \frac{L - \mu_{\widehat{\text{LID}}}}{\sigma_{\widehat{\text{LID}}}}. \tag{15}$$

The mapping function is defined as:

$$\Phi(L) = \alpha_{\min} + \frac{C}{1 + \exp(-z(L))}, \tag{16}$$

where $C = \alpha_{\max} - \alpha_{\min}$. Since we strictly require $\alpha_{\max} > \alpha_{\min}$, it follows that $C > 0$. To determine the sign of the gradient, we apply the chain rule:

$$\frac{d\Phi}{dL} = \frac{d\Phi}{dz} \cdot \frac{dz}{dL}. \tag{17}$$

First, we differentiate the Z-score term with respect to L:

$$\frac{dz}{dL} = \frac{1}{\sigma_{\widehat{\text{LID}}}}. \tag{18}$$

Next, we differentiate the logistic component Φ with respect to z:

$$\frac{d\Phi}{dz} = \frac{d}{dz}\left(\alpha_{\min} + C(1 + e^{-z})^{-1}\right) \tag{19}$$

$$= C \cdot (-1) \cdot (1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) \tag{20}$$

$$= C \cdot \frac{e^{-z}}{(1 + e^{-z})^2}. \tag{21}$$

Combining these terms yields the full derivative:

$$\frac{d\Phi}{dL} = \frac{C}{\sigma_{\widehat{\text{LID}}}} \cdot \frac{e^{-z}}{(1 + e^{-z})^2}. \tag{22}$$

We analyze the sign of each component:

- The operational range constant $C > 0$.
- The standard deviation $\sigma_{\widehat{\text{LID}}} > 0$, assuming the dataset exhibits non-zero geometric variance.
- The exponential function $e^{-z} > 0$ for all $z \in \mathbb{R}$.
- The denominator $(1 + e^{-z})^2 > 0$.

Therefore, the derivative $\frac{d\Phi}{dL} = \frac{C}{\sigma_{\widehat{\text{LID}}}} \frac{e^{-z}}{(1+e^{-z})^2}$ is strictly positive. This confirms that the pruning parameter α strictly increases with the local geometric complexity, thereby enforcing a more conservative pruning that retains more navigational edges in high-LID regions. □

Proposition A.2 (Boundedness)**.** *The pruning parameter $\alpha(u)$ derived from the mapping function is strictly bounded within the prescribed operational interval. For any node u with a finite LID estimate:*

$$\alpha_{\min} < \alpha(u) < \alpha_{\max}. \tag{23}$$

Proof. Let $S(u)$ denote the logistic component of the mapping function:

$$S(u) = \frac{1}{1 + \exp(-z(u))}. \tag{24}$$

For any finite input $\widehat{\mathrm{LID}}(u)$, the Z-score $z(u)$ is finite. The exponential function maps the real line to the positive real line, i.e., $\exp(-z(u)) \in (0, \infty)$. Consequently, the denominator lies in the interval $(1, \infty)$. Taking the reciprocal yields the bounds for the logistic component:

$$0 < S(u) < 1. \tag{25}$$

Substituting $S(u)$ back into the definition of Φ:

$$\alpha(u) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot S(u). \tag{26}$$

Since $(\alpha_{\max} - \alpha_{\min}) > 0$, we can apply the inequality boundaries:

$$\alpha(u) > \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot 0 = \alpha_{\min}, \tag{27}$$

$$\alpha(u) < \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot 1 = \alpha_{\max}. \tag{28}$$

This proves that the topology is strictly confined. The pruning behavior never exceeds the relaxation upper limit ($\alpha_{\max}$) and never becomes stricter than the lower limit ($\alpha_{\min}$), ensuring graph connectivity and preventing degree explosion. □

Lemma A.3 (Local Complexity Lower Bound)**.** *Consistent with the complexity bounds established for growth-restricted metrics [24], for a query q on a manifold $\mathcal{M}$, the expected number of distance evaluations N_{dist} required for successful greedy routing scales exponentially with the local intrinsic dimensionality $d = \mathit{LID}(q)$:*

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\sqrt{d} \cdot \exp(\lambda \cdot d)\right), \tag{29}$$

where $\lambda > 0$ is a geometric constant derived from the required convergence rate.

Proof. Let the local dataset around a query q be modeled as a uniform distribution on a manifold of intrinsic dimension d. Consider a greedy routing step where the algorithm is currently at a node u with distance $r = \|u - q\|$.

1. Geometric Condition for Improvement. To make progress, the algorithm must identify a neighbor v such that the distance to the query is reduced by a factor ϵ (where $0 < \epsilon < 1$), i.e., $\|v - q\| \leq (1 - \epsilon)r$. Let $s = \|u - v\|$ be the distance between the current node and the candidate neighbor (the step size). Applying the Law of Cosines to triangle $\triangle uvq$, we have:

$$\|v - q\|^2 = r^2 + s^2 - 2rs\cos\theta, \tag{30}$$

where θ is the angle between the vector $\vec{uq}$ and $\vec{uv}$. The improvement condition $\|v-q\|^2 \leq (1-\epsilon)^2 r^2$ implies:

$$\begin{aligned} r^2 + s^2 - 2rs\cos\theta &\leq (1-\epsilon)^2 r^2, \\ 2rs\cos\theta &\geq r^2 + s^2 - r^2(1-\epsilon)^2, \\ \cos\theta &\geq \frac{r^2 + s^2 - r^2(1-\epsilon)^2}{2rs}. \end{aligned} \tag{31}$$

Assuming the step size is small relative to the distance (i.e., $s \ll r$), the right-hand side is dominated by $1 - (1-\epsilon)^2 \approx 2\epsilon$, yielding a critical angle threshold θ_{max} such that any successful neighbor must fall within the cone defined by $0 \leq \theta \leq \theta_{max}$.

2. Volume Concentration of the Improving Region. In the tangent space at u, the directions of neighbors are uniformly distributed on the unit hypersphere $\mathbb{S}^{d-1}$. The probability $P_{success}$ of finding a neighbor in the improving cone corresponds to the ratio of the surface area of the spherical cap $\mathcal{C}(\theta_{max})$ to the total surface area of $\mathbb{S}^{d-1}$. This ratio is given by the regularized incomplete beta function, which can be expressed in terms of the integral over the polar angle ϕ:

$$P_{success} = \frac{\text{Area}(\mathcal{C}(\theta_{max}))}{\text{Area}(\mathbb{S}^{d-1})} = \frac{\int_0^{\theta_{max}} \sin^{d-2}\phi \, d\phi}{\int_0^{\pi} \sin^{d-2}\phi \, d\phi}. \tag{32}$$

3. Asymptotic Analysis. We analyze the asymptotic behavior of this ratio as $d \to \infty$. The denominator relates to the surface area of the hypersphere and can be evaluated using the standard integral identity for powers of sine. Exploiting the symmetry of $\sin\phi$ around $\pi/2$, we have:

$$\begin{aligned}\int_0^{\pi} \sin^{d-2}\phi \, d\phi &= 2\int_0^{\frac{\pi}{2}} \sin^{d-2}\phi \, d\phi \\ &= 2 \cdot \frac{\sqrt{\pi}}{2} \frac{\Gamma(\frac{d-1}{2})}{\Gamma(\frac{d}{2})} = \sqrt{\pi}\frac{\Gamma(\frac{d-1}{2})}{\Gamma(\frac{d}{2})}.\end{aligned} \tag{33}$$

For high-dimensional spaces ($d \gg 1$), we apply the asymptotic property of the Gamma function ratio, $\frac{\Gamma(x+a)}{\Gamma(x)} \approx x^a$. Setting $x = d/2$ and $a = -1/2$, we obtain the approximation:

$$\sqrt{\pi}\frac{\Gamma(\frac{d}{2} - \frac{1}{2})}{\Gamma(\frac{d}{2})} \approx \sqrt{\pi}\left(\frac{d}{2}\right)^{-\frac{1}{2}} = \sqrt{\frac{2\pi}{d}}. \tag{34}$$

For the numerator, we exploit the concentration of measure near the integration boundary θ_{max}. Since the integrand $\sin^{d-2}\phi$ decays exponentially fast away from θ_{max}, the integral is dominated by the contribution near the upper limit. We apply integration by parts to extract the leading order term. Recall that $\frac{d}{d\phi}(\sin^{d-1}\phi) = (d-1)\sin^{d-2}\phi\cos\phi$. We rewrite the integrand as:

$$\int_0^{\theta_{max}} \sin^{d-2}\phi \, d\phi = \int_0^{\theta_{max}} \frac{1}{(d-1)\cos\phi}\frac{d}{d\phi}(\sin^{d-1}\phi)\, d\phi. \tag{35}$$

Neglecting lower-order terms generated by the derivative of $\frac{1}{\cos\phi}$, the integral approximates to the boundary term:

$$\left[\frac{\sin^{d-1}\phi}{(d-1)\cos\phi}\right]_0^{\theta_{max}} = \frac{\sin^{d-1}\theta_{max}}{(d-1)\cos\theta_{max}}. \tag{36}$$

Substituting these approximations back into the probability ratio, we obtain:

$$P_{success} \approx \frac{1}{\sqrt{2\pi}\cos\theta_{max}} \cdot \frac{\sqrt{d}}{d-1} \cdot (\sin\theta_{max})^{d-1}. \tag{37}$$

For large d, the term $\frac{\sqrt{d}}{d-1}$ is asymptotically equivalent to $d^{-1/2}$. Since θ_{max} is fixed by the geometric constraints, the trigonometric factors are constant relative to d. Let $\gamma = \sin\theta_{max} < 1$. The probability scales as:

$$P_{success} \propto d^{-1/2} \cdot \gamma^d = d^{-1/2}\exp(d\ln\gamma). \tag{38}$$

Defining the decay rate $\lambda = -\ln\gamma > 0$ (since $\gamma < 1$), we strictly obtain:

$$P_{success} \propto d^{-1/2}\exp(-\lambda \cdot d). \tag{39}$$

4. Complexity Bound. The expected number of distance evaluations follows $\mathbb{E}[N_{dist}] = 1/P_{success}$. Thus:

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\sqrt{d} \cdot \exp(\lambda \cdot d)\right). \tag{40}$$

This confirms that the routing cost is dominated by the exponential term $\exp(\lambda \cdot \mathrm{LID}(q))$, necessitating the adaptive beam width design in MCGI. □

Proposition A.4 (Optimal Budget Allocation)**.** *To maintain a uniform bound on the routing failure probability δ across the manifold (i.e., $\mathbb{P}(\text{fail} \mid q) \leq \delta$ for all $q \in \mathcal{M}$), the search beam width $L(q)$ must scale exponentially with the local intrinsic dimensionality:*

$$L(q) \propto \exp\left(\lambda \cdot \mathit{LID}(q)\right). \tag{41}$$

Proof. Consider a beam search with width L. The probability of failing to find a descent step in one round is approximately $(1 - P_{success})^L$. To guarantee a recall target, we enforce a constant upper bound δ on the failure probability:

$$(1 - P_{success})^L \leq \delta. \tag{42}$$

Taking the natural logarithm and using the approximation $\ln(1-x) \approx -x$ for small $P_{success}$, we require:

$$-L \cdot P_{success} \leq \ln \delta \implies L \geq \frac{-\ln \delta}{P_{success}}. \tag{43}$$

Substituting the relationship $P_{success} = 1/\mathbb{E}[N_{dist}]$ established in Lemma 3.5, we obtain the asymptotic upper bound $P_{success} \leq \mathcal{O}(d^{-1/2} \cdot \exp(-\lambda \cdot d))$, where $d = \text{LID}(q)$. To satisfy the recall constraint in the worst case scenario, the search budget L must grow inversely to $P_{success}$:

$$L \geq \frac{-\ln \delta}{\mathcal{O}(d^{-1/2} \cdot \exp(-\lambda \cdot d))} = \Omega\left(\sqrt{d} \cdot \exp(\lambda \cdot d)\right). \tag{44}$$

In high-dimensional spaces ($d \gg 1$), the exponential growth term dominates the polynomial factor $\sqrt{d}$. By absorbing the lower-order polynomial variations and the confidence factor $-\ln \delta$ into a generalized constant C_δ, the necessary budget asymptotically simplifies to:

$$L(q) \geq C_\delta \cdot \exp(\lambda \cdot \text{LID}(q)). \tag{45}$$

□

Proposition A.5 (Connectivity Preservation)**.** *Let G_{EMST} and G_{RNG} denote the Euclidean Minimum Spanning Tree and the Relative Neighborhood Graph, respectively. For any configuration of points in general position, provided that $\alpha(u) \geq 1.0$ for all $u \in V$, that the occlusion test of each node runs over all other points, and that no degree bound truncates the neighbor lists, the graph G_{MCGI} satisfies the inclusion hierarchy:*

$$E_{EMST} \subseteq E_{RNG} \subseteq E_{MCGI}. \tag{46}$$

Consequently, G_{MCGI} is connected.

Proof. The proof proceeds in two steps. First, we invoke the classical result established by Toussaint [36], which proves that the Relative Neighborhood Graph (RNG) is a supergraph of the Euclidean Minimum Spanning Tree (EMST) for any finite set of points in a metric space:

$$E_{EMST} \subseteq E_{RNG}. \tag{47}$$

Since the EMST connects all vertices in a single component, G_{RNG} is necessarily connected.

Second, we show that $E_{RNG} \subseteq E_{MCGI}$. In the RNG, an edge between u and v is pruned if and only if a witness node falls within the intersection of two spheres centered at u and v with radius $d(u, v)$. We denote this geometric intersection as the exclusion region $\mathcal{R}_{RNG}$. Therefore, an edge exists in the RNG if and only if $\mathcal{R}_{RNG}$ contains no data points. In MCGI, the pruning condition requires the witness node to be within a distance of $\frac{1}{\alpha(u)} d(u, v)$ from v. Because the algorithm enforces $\alpha(u) \geq 1.0$, the pruning radius in MCGI is strictly less than or equal to the pruning radius of the RNG. Consequently, the exclusion region of MCGI is a geometric subset of the RNG exclusion region:

$$\mathcal{R}_{MCGI} \subseteq \mathcal{R}_{RNG}. \tag{48}$$

This subset relationship of exclusion regions creates an inverse superset relationship for the preserved edges. If an edge survives the RNG condition, its $\mathcal{R}_{RNG}$ is empty. Since $\mathcal{R}_{MCGI}$ is contained entirely within $\mathcal{R}_{RNG}$, it must also be empty. Thus, any edge present in the RNG is guaranteed to be preserved in MCGI, yielding the inclusion hierarchy of edge sets:

$$E_{RNG} \subseteq E_{MCGI}. \tag{49}$$

By combining this with the classical result that $E_{EMST} \subseteq E_{RNG}$ derived in [36], we establish the complete global connectivity chain. □

Proposition A.6 (Estimation Robustness)**.** *Let $\widehat{\text{LID}}(u)$ be the MLE estimate computed with k nearest neighbors. The induced perturbation on the pruning parameter satisfies:*

$$|\Delta\alpha(u)| \leq \frac{(\alpha_{\max} - \alpha_{\min}) \cdot \text{LID}(u)}{4\, \sigma_{\widehat{\text{LID}}} \sqrt{k}}. \tag{50}$$

Proof. Let $\epsilon = \widehat{\mathrm{LID}}(u) - \mathrm{LID}(u)$ denote the estimation error. By the first-order Taylor expansion of Φ around $\mathrm{LID}(u)$:

$$|\Delta\alpha(u)| = \left|\Phi\Big(\widehat{\mathrm{LID}}(u)\Big) - \Phi(\mathrm{LID}(u))\right| \leq \sup_L \left|\frac{d\Phi}{dL}\right| \cdot |\epsilon|. \tag{51}$$

From the derivative computed in Proof A.1, the magnitude of the gradient is:

$$\left|\frac{d\Phi}{dL}\right| = \frac{C}{\sigma_{\widehat{\mathrm{LID}}}} \cdot \frac{e^{-z}}{(1+e^{-z})^2}, \tag{52}$$

where $C = \alpha_{\max} - \alpha_{\min}$. The logistic factor $\frac{e^{-z}}{(1+e^{-z})^2}$ achieves its global maximum of $1/4$ at $z = 0$, so:

$$\left|\frac{d\Phi}{dL}\right| \leq \frac{\alpha_{\max} - \alpha_{\min}}{4\,\sigma_{\widehat{\mathrm{LID}}}}. \tag{53}$$

Substituting the asymptotic error bound of the MLE estimator, whose variance is of order LID^2/k, namely $|\epsilon| = O(\mathrm{LID}(u)/\sqrt{k})$ [26], yields:

$$|\Delta\alpha(u)| \leq \frac{(\alpha_{\max} - \alpha_{\min}) \cdot \mathrm{LID}(u)}{4\,\sigma_{\widehat{\mathrm{LID}}}\,\sqrt{k}}. \tag{54}$$

This completes the proof. □

Proposition A.7 (Suboptimality of Fixed Beam)**.** *For any fixed beam width $\bar{L}$, the success probability at a query q satisfies Eq.* (12)*, which decays exponentially with* $\mathrm{LID}(q)$.

Proof. Consider a beam search with fixed width $\bar{L}$. From Proof A.4, the failure probability at query q is

$$P(\text{fail} \mid q) = (1 - P_{\text{success}}(q))^{\bar{L}}. \tag{55}$$

Bernoulli's inequality $(1-x)^n \geq 1 - nx$ for $x \in [0,1]$ and integer $n \geq 1$ gives

$$1 - P(\text{fail} \mid q) \leq \bar{L}\, P_{\text{success}}(q). \tag{56}$$

Definition 3.8 sets $L^*(q) = \ln(1/\delta)/P_{\text{success}}(q)$, hence $P_{\text{success}}(q) = \ln(1/\delta)/L^*(q)$, and substitution yields

$$1 - P(\text{fail} \mid q) \leq \ln(1/\delta)\,\frac{\bar{L}}{L^*(q)} = \frac{\ln(1/\delta)\,\bar{L}}{C_\delta}\,\exp(-\lambda \cdot \mathrm{LID}(q))\,, \tag{57}$$

where the last step uses $L^*(q) = C_\delta \exp(\lambda \cdot \mathrm{LID}(q))$. The right-hand side decays exponentially with $\mathrm{LID}(q)$, and it falls below the target $1-\delta$ exactly when $L^*(q) > \bar{L}\ln(1/\delta)/(1-\delta)$. □

Corollary A.8 (Construction-time Structural Compensation)**.** *Under any fixed beam width $\bar{L}$ and without the degree bound R, the adaptive pruning in MCGI monotonically reduces $P(\text{fail} \mid q)$ for every query whose beam search visits only high-LID nodes, compared to any static baseline $\bar{\alpha} \leq (\alpha_{\min} + \alpha_{\max})/2$ operating under the same $\bar{L}$.*

Proof. We proceed in two steps: first showing that MCGI preserves a strictly larger edge set in high-LID regions, then connecting this to a lower failure probability under fixed beam search.

Step 1: Edge set inclusion. The pruning criterion in MCGI discards an edge (u, v) if and only if there exists a witness node n such that $\alpha(u) \cdot d(n, v) \leq d(u, v)$. For nodes where $\mathrm{LID}(u) > \mathrm{LID}_{\text{avg}}$, Proposition A.1 guarantees $\alpha(u) > \Phi(\mathrm{LID}_{\text{avg}}) = (\alpha_{\min} + \alpha_{\max})/2 \geq \bar{\alpha}$. Since $\alpha(u) > \bar{\alpha}$ strictly enlarges the left-hand side of the pruning condition, any edge that would be pruned under $\alpha(u)$ is also pruned under $\bar{\alpha}$, but not vice versa, for the same candidate pool and the same witness nodes. The degree bound R is not applied in this step; with the bound, both edge sets are truncated to at most R edges, and the inclusion holds only for the edges that survive the occlusion test. Therefore $\mathcal{E}_{\text{MCGI}} \supseteq \mathcal{E}_{\bar{\alpha}}$ in high-LID regions, with strict inclusion whenever at least one edge survives $\alpha(u)$ but not $\bar{\alpha}$.

Step 2: Failure probability. Under beam search with fixed width $\bar{L}$, the failure probability at query q is governed by $P_{\text{success}}(q)$, the probability of finding a descent step from any node on the routing path (Proof A.4). A strictly larger outgoing edge set at each high-LID node monotonically increases

the number of candidate directions evaluated per beam step, which by the same geometric argument in Proof A.4 yields a weakly higher $P_{\text{success}}(q)$. Therefore, for every query q whose beam search visits only nodes with $\text{LID}(u) > \text{LID}_{\text{avg}}$,

$$P(\text{fail} \mid q, \text{MCGI}) \leq P(\text{fail} \mid q, \bar{\alpha}), \tag{58}$$

with equality only when $\alpha(u) = \bar{\alpha}$ for all u on the routing path, i.e., when the data is geometrically uniform. This establishes a monotone improvement over fixed-α baselines; quantifying the residual gap to the Iso-Recall Schedule $L^*(q)$ (Definition 3.8) is left for future work. □

A.2 Additional Experimental Details

Experiment Platform All experiments run on a cloud virtual machine with 192 virtual cores of Intel Xeon Platinum 8468 and 960 GiB of RAM. The operating system is Ubuntu 24.04 LTS, and all code is compiled with GCC 13.3 at optimization level -O3. The disk-resident indexes reside on a 17.6 TB virtual block device (virtio) of the virtual machine, which the DiskANN search engine reads through Linux asynchronous I/O with direct I/O, so that the operating system page cache does not serve graph nodes. Every search uses 32 threads and a beam width of 2, and the page cache is dropped before each search run. All indexes are built on the CPU cores with the DiskANN builder; the SIFT1B and Deep1B builds use a memory budget of 300 GiB and 192 threads.

Datasets We evaluate our method on three standard million-scale benchmarks whose dimensionality ranges from 100 to 960 and on two billion-scale benchmarks, so that the datasets cover both high ambient dimensionality and large scale. In the following descriptions, N denotes the number of base vectors and D represents the vector dimensionality:

- SIFT1M [22] ($N = 10^6, D = 128$): Local image descriptors with 10,000 queries, searched with Euclidean distance (L_2).
- GloVe-100 [30] ($N = 1.18 \times 10^6, D = 100$): Word embedding vectors with 10,000 queries. Following standard practice, we normalize the vectors to unit length and use Euclidean distance as a proxy for cosine similarity.
- GIST1M [22] ($N = 10^6, D = 960$): Global image descriptors with 1,000 queries, which have the highest ambient dimensionality and the widest spread of LID values among the million-scale datasets.
- SIFT1B [23] ($N = 10^9, D = 128$): One billion local image descriptors stored as 8-bit unsigned integers, with 10,000 queries and the official ground truth over the full billion vectors.
- Deep1B [2] ($N = 10^9, D = 96$): One billion image descriptors from a convolutional neural network, reduced to 96 dimensions by PCA, normalized to unit length, and stored as 32-bit floats, with 10,000 queries. The official ground truth over the full billion vectors is wrong for three queries (indices 449, 2382, and 2809)[1], which we exclude.

The calibration queries of the query-adaptive search list are disjoint from the evaluation queries. For SIFT1M and GIST1M, they are the first 10,000 vectors of the learning set of each dataset. GloVe-100 has no learning set; its calibration queries are 10,000 base vectors drawn with a fixed seed, and each of them is excluded from its own ground truth. The ground truth of every calibration query consists of its exact 100 nearest neighbors. For SIFT1B and Deep1B, the calibration queries are the even-indexed half of the official queries, and the evaluation uses the odd-indexed half, which leaves 5,000 evaluation queries on SIFT1B and 4,998 on Deep1B.

Baselines We compare MCGI against DiskANN (Vamana) [34], the standard disk-resident graph index. Both methods search the same index file with the same compressed vectors, the same beam width, and the same requested list sizes. The comparison therefore isolates the query-adaptive search list of MCGI, since the index, the I/O path, and the distance computation are identical. On GIST1M, the following baselines complement DiskANN, and each of them searches the 1,000 official queries with 32 threads once per setting.

[1] `https://github.com/harsha-simhadri/big-ann-benchmarks/issues/353`

Table 2: Dataset specifications and indexing parameters.

Dataset	N	D	Queries	R	L_{build}	m_{PQ} (Byte)	Data Type
SIFT1M	10^6	128	10,000	32	50	107	float32
GloVe-100	1.18×10^6	100	10,000	64	100	90	float32
GIST1M	10^6	960	1,000	32	100	322	float32
SIFT1B	10^9	128	5,000	32	50	31	uint8
Deep1B	10^9	96	4,998	32	50	31	float32

Table 3: LID statistics and search parameters.

Dataset	μ_{LID} (Mean)	σ_{LID} (Std)	List size $\bar{L}$	γ	Threads
SIFT1M	16.1	9.2	$10 \rightarrow 400$	0.7	32
GloVe-100	35.3	18.1	$10 \rightarrow 400$	1.0	32
GIST1M	48.9	25.9	$10 \rightarrow 400$	0.7	32
SIFT1B	–	–	$10 \rightarrow 1000$	0.7	32
Deep1B	–	–	$10 \rightarrow 2000$	0.7	32

- PipeANN [12, 13] (pipelined search with `io_uring`) searches the DiskANN index files of MCGI with requested list sizes from 10 to 400 and I/O widths 4 and 32, of which width 4 gives the higher throughput and enters Figure 2. Its entry-point index is built from a 1% sample of the base vectors, and its per-vector PQ limit is raised from 256 to 512 bytes so that it loads the 322-byte codes of the GIST1M indexes.
- CSPG [39] uses the official implementation with the settings of its authors for GIST1M, namely two partitions of Vamana subgraphs, a construction list of 128, $\alpha = 1.2$, and a duplication ratio of 0.5, with $R = 32$ and $R = 48$. Its parallel search assigns queries to threads in blocks of 64, so that the 1,000 GIST1M queries occupy at most 16 of the 32 threads.
- HNSW [29] uses hnswlib with $M = 16$, which bounds the degree of the base layer by 32, and a construction list of 200.
- IVF-Flat [21] uses Faiss with 4,096 lists and full-precision vectors.

The QPS of every method at a target recall is interpolated along its measured curve.

Evaluation Metrics We adhere to the standard evaluation protocol for ANN search and measure Recall@10 against QPS (queries per second). Each method yields one Recall@10 and QPS point per requested list size. The throughput gain at a target recall is the ratio of the QPS of the two methods, where each QPS value is linearly interpolated along the recall axis of its curve.

Build Configuration Table 2 summarizes the dataset specifications and the indexing parameters of the index that both methods search, where the query count is the number of evaluation queries. R denotes the maximum outdegree of each node in the graph, and L_{build} is the size of the candidate list during index construction. Every index uses the standard pruning parameter $\alpha = 1.2$ of DiskANN. m_{PQ} specifies the number of bytes of the Product Quantization (PQ) code of each vector, which follows from the PQ memory budget of the build.

Search Configuration Table 3 lists the LID statistics of each dataset and the parameters of the query-adaptive search list. μ_{LID} and σ_{LID} denote the mean and standard deviation of the maximum-likelihood LID estimates of the base vectors with 20 neighbors, which we compute for the million-scale datasets only. The checkpoint of Eq. (13) follows $n_0 = 24$ expansions, the candidate-list estimator uses the $m = 64$ best candidates, and the competition count uses $\epsilon = 0.2$. The list size is bounded by $L_{\min} = 10$ and $c = 8$, and the strength γ is chosen from $\{0.5, 0.7, 1.0, 1.3\}$ on the calibration queries of each dataset.

Latency Figure 4 plots the recall of both methods against the requested list size $\bar{L}$ on GIST1M and the mean query latency against recall on GIST1M, SIFT1B, and Deep1B. The recall of MCGI closely

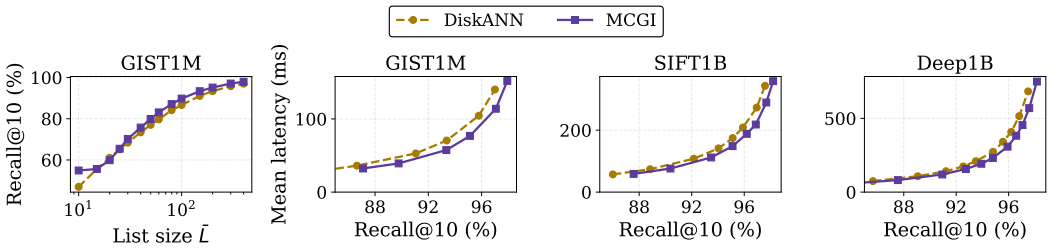


Figure 4: Recall against the requested list size, and mean latency against recall.

Table 4: Offline cost in seconds: index build stages and calibration of the query-adaptive search list.

Dataset	PQ training	Graph	Disk layout	Total build	Calibration
SIFT1M	195	6	1	203	174
GloVe-100	400	15	1	416	168
GIST1M	1,678	82	4	1,764	174
SIFT1B	2,775	14,117	290	17,225	94
Deep1B	3,373	15,029	836	19,317	289

follows that of DiskANN at every requested list size, so $\bar{L}$ remains the single knob of the search. At matched recall, MCGI lowers the mean latency, at 95% recall from 93 to 75 ms on GIST1M and from 290 to 245 ms on Deep1B, and at 90% recall from 86 to 74 ms on SIFT1B.

Offline Cost Table 4 lists the offline cost of each index, where every build runs alone on the platform with 192 threads and the calibration runs alone with 32 search threads. For the million-scale datasets, the table reports a separate rebuild with the same parameters, because the evaluated indexes were built while other jobs shared the machine. The training of the PQ codebooks dominates the build of the million-scale indexes, whereas the graph construction over the shards and their merge dominates at one billion vectors. The only offline step that MCGI adds is the calibration of Eq. (13), which searches the calibration queries once at each list size of a fixed grid and then fits the weights by ridge regression within seconds. The grid has 14 list sizes from 10 to 400 for the million-scale datasets and SIFT1B, and 18 list sizes from 10 to 1,000 for Deep1B. Calibration takes about 3 minutes on each million-scale dataset, and 1.6 and 4.8 minutes on SIFT1B and Deep1B, which is at most 1.5% of the build time of a billion-scale index.

A.3 Limitations

The throughput gains in Section 4 come from the query-adaptive search list, which runs on indexes built with the standard Vamana pruning rule. The construction-time rule of Section 3.1 relaxes the occlusion test of high-LID nodes; under the degree bound R of a disk-resident index, however, a node fills its neighbor list either way, so that the rule changes which candidate edges a node keeps rather than how many. Consequently, Corollary 3.10 and Proposition 3.7 hold without the degree bound, and the evaluation attributes no gain to the construction-time rule.

The query-adaptive list also has costs that its mean throughput does not show. Because hard queries receive lists up to c times longer than $\bar{L}$, the 99.9th-percentile latency at matched recall rises, for example from 119 to 181 ms on GIST1M at 95% recall, while the mean latency falls. The calibration further needs a few thousand queries from the query distribution together with their exact nearest neighbors, and the fitted weights apply to the index on which they were fitted. Finally, the analysis of Section 3.3 models each beam step as an independent trial whose success probability follows from a uniform local model of the data, so that its bounds describe trends rather than exact recall values.

A.4 Algorithm Pseudo-Code

Algorithm 1 outlines the offline construction process of the indexing structure. The procedure is divided into two sequential phases.

- The first phase focuses on geometric calibration by calculating the Local Intrinsic Dimensionality for all vectors in the dataset in parallel. After obtaining the global mean and standard deviation of these estimates, the algorithm assigns a personalized pruning parameter α_u to each node. This parameter is derived using a logistic function that maps the standardized geometric complexity of the local neighborhood into a bounded operational range.
- The second phase handles topology refinement. Starting from a random graph, the algorithm iteratively optimizes the connections. For each node, it executes a greedy search to identify a pool of candidate neighbors. It then evaluates these candidates alongside existing neighbors using a dynamic occlusion criterion. A candidate is pruned if a previously accepted neighbor falls within a restricted exclusion region defined by the precomputed α_u. Survivors are added to the updated neighbor list until the degree limit is reached.

Algorithm 1 Manifold-Consistent Graph Indexing (MCGI)

Input: Dataset X, Max Degree R, Beam Width L
Output: Optimized Graph G

// Phase 1: Geometric Calibration
$\mathcal{L} \leftarrow$ ParallelEstimateLID(X)
$\mu \leftarrow$ Mean($\mathcal{L}$)
$\sigma \leftarrow$ StdDev($\mathcal{L}$)
for each node $u \in V$ in parallel **do**
  $z_u \leftarrow (\mathcal{L}[u] - \mu)/\sigma$
  $\alpha_u \leftarrow \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})/(1 + \exp(-z_u))$
end for

// Phase 2: Topology Refinement
$G \leftarrow$ RandomGraph(X, R)
for $iter \leftarrow 1$ to $MaxIter$ **do**
  for each node $u \in G$ in parallel **do**
    $\mathcal{C} \leftarrow$ GreedySearch(u, G, L)
    $\mathcal{N}_{new} \leftarrow \emptyset$
    for $v \in$ SortByDistance($\mathcal{C} \cup \mathcal{N}(u)$) **do**
      $pruned \leftarrow$ False
      for $n \in \mathcal{N}_{new}$ **do**
        if $\alpha_u \cdot d(n, v) \leq d(u, v)$ **then**
          $pruned \leftarrow$ True; break
        end if
      end for
      if not $pruned$ **and** $|\mathcal{N}_{new}| < R$ **then**
        $\mathcal{N}_{new}$.add(v)
      end if
    end for
    $\mathcal{N}(u) \leftarrow \mathcal{N}_{new}$
  end for
end for

Algorithm 2 presents an optimized online variant designed to handle massive datasets where exhaustive geometric profiling is computationally prohibitive.

- It begins with a fast approximation phase that samples a small subset of the data. By evaluating this subset, the algorithm quickly bootstraps the global mean and standard deviation of the underlying probability distribution. This step provides a coarse but efficient estimate of the overall geometric complexity, enabling the algorithm to initialize the pruning parameters without incurring the cost of a full dataset pass.
- The second phase merges the geometric estimation directly into the iterative topology refinement loop. Instead of computing all complexities beforehand, the algorithm dynamically estimates the node specific dimensionality on the fly using the candidate set discovered during the greedy search step. This real time estimate is immediately used to calculate the

specific pruning parameter α_u. The algorithm then applies this dynamic parameter to the pruning criterion, determining which edges to preserve and which to discard. This approach ensures the graph connectivity adapts to the manifold geometry progressively as the quality of the candidate neighbors improves with each iteration.

Algorithm 2 Online Manifold-Consistent Graph Indexing

Input: Dataset X, Max Degree R, Beam Width L, Sample Rate S
Output: Optimized Graph G

// Phase 1: Bootstrap Statistics (Fast Approximation)
$X_{sample} \leftarrow$ RandomSample(X, S)
$\mathcal{L}_{sample} \leftarrow$ EstimateLID(X_{sample}) {Compute LID on subset}
$\mu \leftarrow$ Mean$(\mathcal{L}_{sample})$, $\sigma \leftarrow$ StdDev$(\mathcal{L}_{sample})$

// Phase 2: Online Topology Refinement
$G \leftarrow$ RandomGraph(X, R)
for $iter \leftarrow 1$ to $MaxIter$ **do**
  for each node $u \in G$ in parallel **do**
    $\mathcal{C} \leftarrow$ GreedySearch(u, G, L)
    // Online LID Estimation & Parameter Adaptation
    $\hat{\ell}_u \leftarrow$ ComputeLID$(u, \mathcal{C})$ {Estimate from neighbors}
    $z_u \leftarrow (\hat{\ell}_u - \mu)/\sigma$
    $\alpha_u \leftarrow \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})/(1 + \exp(-z_u))$
    $\mathcal{N}_{new} \leftarrow \emptyset$
    for $v \in$ SortByDistance$(\mathcal{C} \cup \mathcal{N}(u))$ **do**
      $pruned \leftarrow$ False
      for $n \in \mathcal{N}_{new}$ **do**
        // Pruning with dynamic α_u
        if $\alpha_u \cdot d(n, v) \leq d(u, v)$ **then**
          $pruned \leftarrow$ True; break
        end if
      end for
      if not $pruned$ **and** $|\mathcal{N}_{new}| < R$ **then**
        $\mathcal{N}_{new}$.add(v)
      end if
    end for
    $\mathcal{N}(u) \leftarrow \mathcal{N}_{new}$
  end for
end for